

ESP32 Smart Auto / Manual Lighting System

Ahmad Azroun

Renewable Energy Manager | IoT & Smart Energy Systems Developer

Projektbeschreibung

Dieses Projekt demonstriert ein intelligentes adaptives Beleuchtungssystem mit ESP32, LDR-Lichtsensord, PIR-Bewegungssensord, PWM-Technologie und manueller Steuerung. Das System analysiert kontinuierlich die Umgebungshelligkeit und erkennt Bewegungen in Echtzeit. Bei heller Umgebung bleiben die LEDs ausgeschaltet. Bei Dunkelheit aktiviert das System automatisch die Beleuchtung. Wenn eine Bewegung erkannt wird, wechseln die LEDs sanft zur maximalen Helligkeit. Nach Ablauf des Timers dimmen die LEDs automatisch zurück in den Energiesparmodus. Zusätzlich wurde ein Push-Button integriert, um zwischen drei Betriebsmodi umzuschalten: AUTO MODE, MANUAL ON und MANUAL OFF.

Funktion	Beschreibung
LDR Sensor	Misst die Lichtintensität
PIR Sensor	Erkennt Bewegungen
PWM Fade	Sanftes Ein- und Ausblenden
AUTO MODE	Automatische intelligente Steuerung
MANUAL ON	LEDs dauerhaft eingeschaltet
MANUAL OFF	LEDs dauerhaft ausgeschaltet
Push Button	Wechsel zwischen Betriebsmodi

Nutzen des Projekts

Dieses intelligente Beleuchtungssystem kann in Smart Homes, intelligenter Straßenbeleuchtung, Solarbeleuchtungssystemen, Garagen, Sicherheitsanlagen, Lagerräumen und energieeffizienten Gebäuden verwendet werden. Das Projekt zeigt, wie intelligente IoT-Systeme den Energieverbrauch reduzieren können, indem starke Beleuchtung nur bei Bedarf aktiviert wird.

MicroPython Code

```
from machine import Pin, ADC, PWM
import time

leds = PWM(Pin(14))
leds.freq(10000)

internal_led = Pin(2, Pin.OUT)

light_sensor = ADC(Pin(34))
light_sensor.atten(ADC.ATTN_11DB)
light_sensor.width(ADC.WIDTH_12BIT)

motion_sensor = Pin(27, Pin.IN)
```

```
button = Pin(25, Pin.IN, Pin.PULL_UP)

motion_timeout = 5
last_motion_time = 0

current_brightness = 0

mode = 0
last_button_state = 1

def fade_to(target):
    global current_brightness

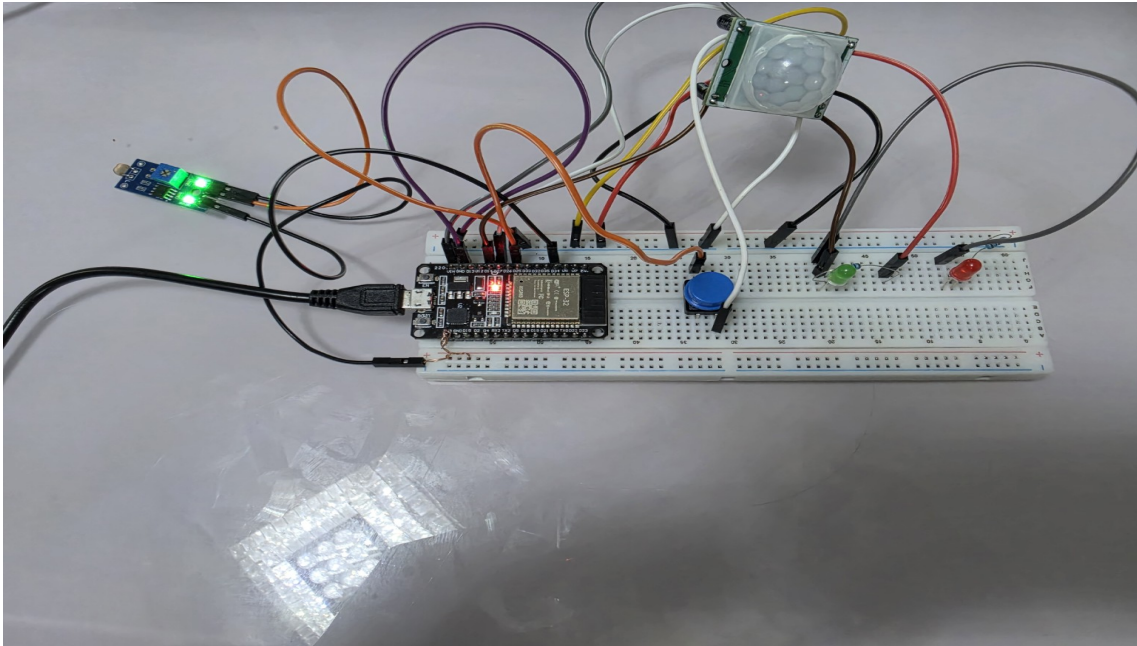
    if target > current_brightness:
        for value in range(current_brightness, target + 1, 20):
            leds.duty(value)
            time.sleep(0.05)

    elif target < current_brightness:
        for value in range(current_brightness, target - 1, -20):
            leds.duty(value)
            time.sleep(0.05)

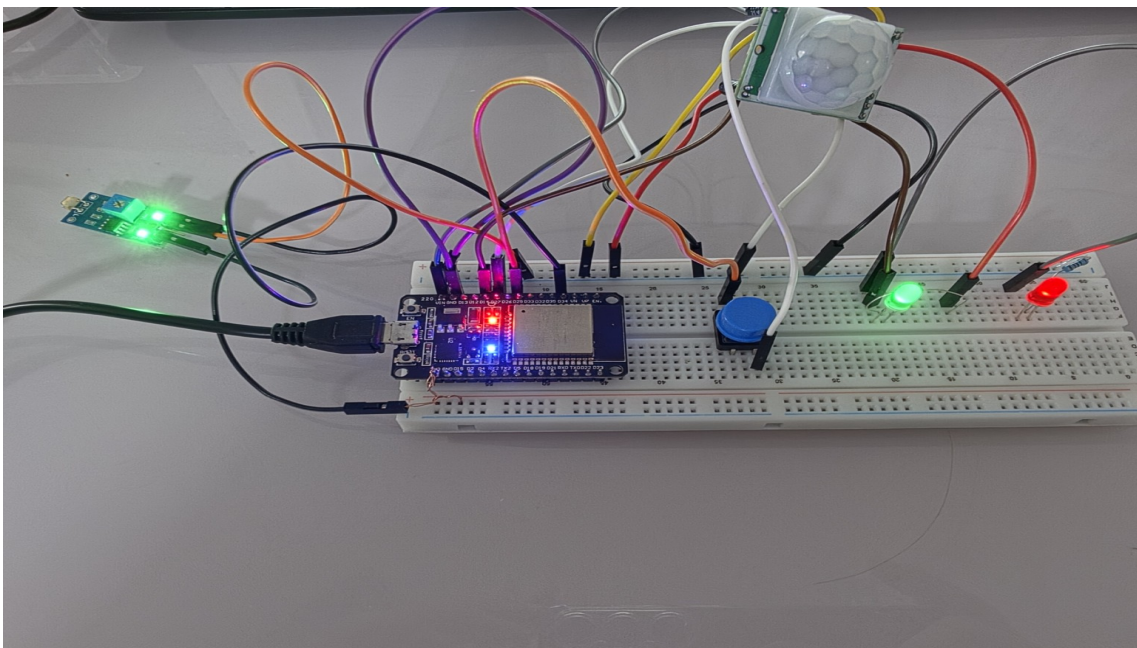
    current_brightness = target
    leds.duty(current_brightness)
```

Projektbilder

Helle Umgebung → LEDs AUS



Dunkelheit + Bewegung → LEDs EIN



ESP32 Smart Auto / Manual Lighting System

Ahmad Azroun

Renewable Energy Manager | IoT & Smart Energy Systems Developer

Project Description

This project demonstrates an intelligent adaptive lighting system using ESP32, an LDR light sensor, a PIR motion sensor, PWM technology, and manual control. The system continuously analyzes ambient light intensity and detects movement in real time. In bright environments, the LEDs remain OFF. In dark environments, the system automatically activates the lighting. When motion is detected, the LEDs smoothly fade to maximum brightness. After the timeout period, the LEDs automatically dim back to energy-saving mode. A push button was also integrated to switch between three operating modes: AUTO MODE, MANUAL ON, and MANUAL OFF.

Feature	Description
LDR Sensor	Measures ambient light intensity
PIR Sensor	Detects movement
PWM Fade	Smooth fade-in and fade-out
AUTO MODE	Automatic intelligent control
MANUAL ON	LEDs permanently ON
MANUAL OFF	LEDs permanently OFF
Push Button	Switch between operating modes

Project Benefits

This intelligent lighting system can be used in smart homes, smart street lighting, solar lighting systems, garages, security systems, warehouses, and energy-efficient buildings. The project demonstrates how intelligent IoT systems can reduce energy consumption by activating strong lighting only when required.

MicroPython Code

```
from machine import Pin, ADC, PWM
import time

leds = PWM(Pin(14))
leds.freq(1000)

internal_led = Pin(2, Pin.OUT)

light_sensor = ADC(Pin(34))
light_sensor.atten(ADC.ATTN_11DB)
light_sensor.width(ADC.WIDTH_12BIT)

motion_sensor = Pin(27, Pin.IN)

button = Pin(25, Pin.IN, Pin.PULL_UP)
```

```
motion_timeout = 5
last_motion_time = 0

current_brightness = 0

mode = 0
last_button_state = 1

def fade_to(target):
    global current_brightness

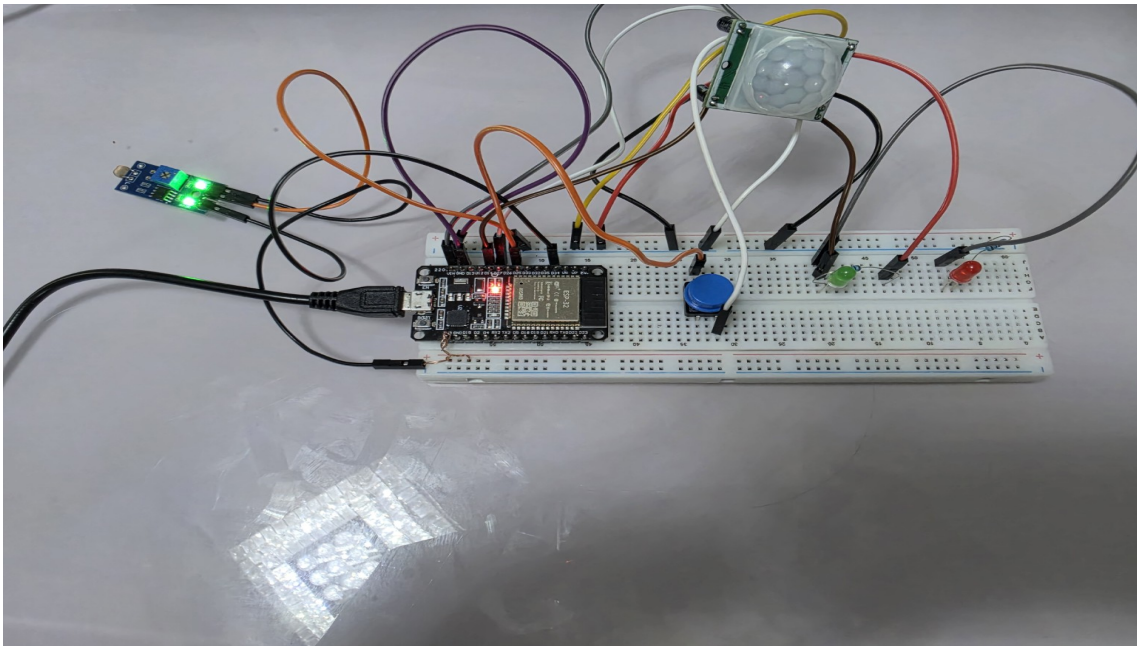
    if target > current_brightness:
        for value in range(current_brightness, target + 1, 20):
            leds.duty(value)
            time.sleep(0.05)

    elif target < current_brightness:
        for value in range(current_brightness, target - 1, -20):
            leds.duty(value)
            time.sleep(0.05)

    current_brightness = target
    leds.duty(current_brightness)
```

Project Images

Bright Environment → LEDs OFF



Dark Environment + Motion → LEDs ON

